

153

Output

```
Prime Thread: 153 is not a prime
number
Armstrong Thread: 153 is an
Armstrong number
Reverse Thread: Reverse of 153 =
351
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

```
1 import java.util.Scanner;
2 public class ThreadA extends Thread{
3     static int n;
4     public void run() {
5         if(getName().equals("Prime")){
6             boolean p=true;
7             for(int i=1;i<n;i++){
8                 if(n%i==0){
9                     p=false;
10                    break;
11                }
12            }
13            if(p)
14                System.out.println("Prime Thread: "+n+" is a prime number");
15            else{
16                System.out.println("Prime Thread: "+n+" is not a prime number");
17            }
18        }
19        else if(getName().equals("Armstrong")){
20            int t=n;
21            int s=0;
22            int d=String.valueOf(n).length();
23            while(t>0){
24                int dig=t%10;
25                s+=Math.pow(dig,d);
26                t/=10;
27            }
28            if(s==n){
```

```
28         if(s==n){
29             System.out.println("Armstrong Thread: "+n+" is an Armstrong number");
30         }
31         else
32             System.out.println("Armstrong Thread: "+n+" is not an Armstrong number");
33     }
34     else{
35         int t=n;
36         int r=0;
37         while(t>0){
38             int d=t%10;
39             r=r*10+d;
40             t/=10;
41         }
42         System.out.println("Reverse Thread: Reverse of "+n+" = "+r);
43     }
44 }
45 public static void main(String[] args){
46     Scanner sc= new Scanner(System.in);
47     if(sc.hasNextInt()){
48         n=sc.nextInt();
49     }
50     ThreadA t1=new ThreadA();
51     ThreadA t2 = new ThreadA();
52     ThreadA t3=new ThreadA();
53     t1.setName("Prime");
54     t2.setName("Armstrong");
55     t3.setName("Reverse");
```

```
56     try{
57         t1.start();
58         t2.start();
59         t3.start();
60         t1.join();
61         t2.join();
62         t3.join();
63     }catch(InterruptedException e){
64         e.printStackTrace();
65     }
66 }
67 }
```

```

1 import java.util.Scanner;
2 public class Main{
3     public static void main(String args[]){
4         Scanner sc=new Scanner(System.in);
5         int n=sc.nextInt();
6         PrimeThread pt=new PrimeThread(n);
7         FactorialThread ft=new FactorialThread(n);
8         ReverseThread rt=new ReverseThread(n);
9         try{
10             pt.start();
11             pt.join();
12             ft.start();
13             ft.join();
14             rt.start();
15             rt.join();
16         }catch(InterruptedException e){
17             e.printStackTrace();
18         }
19     }
20 }
21 }
22 class PrimeThread extends Thread {
23     int number;
24
25     PrimeThread(int number) {
26         this.number = number;
27     }
28
29     public void run() {
30         boolean isPrime = true;
31         if (number < 2) {
32             isPrime = false;
33         } else {
34             for (int i = 2; i <= Math.sqrt(number); i++) {
35                 if (number % i == 0) {
36                     isPrime = false;
37                     break;
38                 }
39             }
40         }
41         if (isPrime) {
42             System.out.println("Prime: " + number + " is a prime number");
43         } else {
44             System.out.println("Prime: " + number + " is not a prime number");
45         }
46     }
47 }
48
49 class FactorialThread extends Thread {
50     int number;
51
52     FactorialThread(int number) {
53         this.number = number;
54     }
55
56     public void run() {
57         long factorial = 1;
58         for (int i = 1; i <= number; i++) {
59             factorial *= i;
60         }
61         System.out.println("Factorial: " + number + "! = " + factorial);
62     }
63 }
64
65 class ReverseThread extends Thread {
66     int number;
67
68     ReverseThread(int number) {
69         this.number = number;
70     }
71
72     public void run() {
73         int reversed = 0;
74         int temp = number;
75         while (temp != 0) {
76             int digit = temp % 10;
77             reversed = reversed * 10 + digit;
78             temp /= 10;
79         }
80         System.out.println("Reverse: Reverse of " + number + " = " + reversed);
81     }
82 }

```

7

Output

```

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

20.00 / 20 marks

Your Code

```
1 public class Thread1{
2     public static void main(String args[]){
3         increment i=new increment();
4         ThreadA t1=new ThreadA(i);
5         ThreadB t2=new ThreadB(i);
6         try{
7             t1.start();
8             t1.join();
9             t2.start();
10            t2.join();
11
12        }catch(InterruptedException e){
13            e.printStackTrace();
14        }
15        System.out.println("Final Count = "+increment.c);
16    }
17 }
18 class increment{
19     static int c=0;
20     public void count(){
21         c++;
22     }
23 }
24 class ThreadA extends Thread{
25     increment in;
26     public ThreadA(increment in){
27         this.in=in;
28     }
29     public void run(){
30         for(int i=0;i<1000;i++){
31             in.count();
32         }
33         System.out.println("ThreadA completed");
34     }
35 }
36 class ThreadB extends Thread{
37     increment inc;
38     public ThreadB(increment inc){
39         this.inc=inc;
40     }
41     public void run(){
42         for (int i=0;i<1000;i++){
43             inc.count();
44         }
45         System.out.println("ThreadB completed");
46     }
47 }
```

Submitted

Input

stdin input

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks